ARGUE / 01

Let Codex challenge the plan.

Claude writes it. Codex reviews it. You decide what survives.

Start here

Open ARGUE-Prompt.txt and copy the marked prompt into Claude Code with your project folder open. Replace [TASK] and [PROJECT] first. Use a small, trusted project for the first run.

The review loop

1. Claude writes plan-v1.md with a clear task and acceptance tests.
2. Codex inspects the plan and relevant files read-only.
3. Claude accepts, rejects with evidence, or marks each finding unresolved.
4. Codex checks the revised plan once. You review the remaining decisions.

Keep a stopping point

This version stops after two review rounds. Stop earlier if no substantiated issue remains. Agreement can still be wrong: run the acceptance tests and check the evidence before building.

Before you run it

Both tools must be installed, signed in, and available in the same local project environment. Ordinary browser chat cannot invoke your local CLI. The free download does not include Claude or Codex usage; each review uses your applicable allowance or API billing.

What this download is

A copy-ready adaptation of the planning-and-critique workflow from my Use Both video and existing public kit. The added stop rule and decision log make it practical to reuse. It is not a verbatim export of a private session.

The copy-ready prompt

Editable text version included: ARGUE-Prompt.txt

Create a plan for [TASK] in [PROJECT]. Do not implement it yet.

Write plan-v1.md with the goal, scope, assumptions, steps, acceptance tests, and risks. Read the relevant project instructions and files first. Keep secrets and unrelated private data out of the review.

Then use the installed Codex CLI as an independent reviewer. No extra plugin or skill is needed for this workflow. Give Codex the same task brief and plan-v1.md. Ask it to inspect the relevant project files read-only and look for incorrect assumptions, missing requirements, edge cases, security issues, and tests that would actually prove the plan works. Ask it to cite evidence and suggest the smallest useful fix for each finding.

Use this command shape, adapting only the task brief and relevant file names:

codex exec --sandbox read-only --output-last-message review-round-1.md "Read plan-v1.md and the relevant project instructions. Independently review this plan against its task brief and acceptance tests. Do not edit project files. Report concrete issues with evidence, impact, and the smallest useful fix. Mark speculation clearly. If you find no substantiated issue, say so without inventing one."

Read its critique. For each finding, either accept it and revise the plan, reject it with specific evidence, or mark it unresolved. Save plan-v2.md and a short decision log.

Send the revised plan and decision log back to Codex for one final read-only review. Use review-round-2.md as the output file. Ask it to check whether the fixes address its findings and whether any material issue remains. Stop after these two review rounds, or sooner if there are no substantiated issues left.

Finish with the final plan, accepted changes, unresolved disagreements, and the tests or evidence a human must check before implementation. Agreement alone is not proof. Do not force agreement, invent a clean bill of health, or implement anything until I approve the plan.

If Codex is unavailable, cannot authenticate, or cannot access the project, stop and explain the exact blocker. Give me a copy-ready review prompt to run manually. Do not install tools, switch accounts, broaden permissions, or pretend the review ran.

ARGUE / 03

Check the result.

A specific challenge beats a confident thumbs-up.

A small practice task

Plan a CSV export for a fictional task tracker. Exclude private notes. Require tests for authorized records only, commas and line breaks, and spreadsheet formula injection. Review a plan before writing application code.

A useful review sounds like this

"The export includes note text even though the brief excludes private notes. Remove that field and test the CSV headers and each exported row."

The finding identifies a requirement, a defect, and a check. Treat unsupported claims as unresolved; do not turn them into facts.

If the CLI cannot run

Check claude --version, codex --version, and codex exec --help in your terminal. Sign in through each tool normally. Do not disable permissions to get past an error.

Manual route: give Claude the task, paste its plan into Codex, then paste the critique back into Claude. Repeat once. Keep the brief and plan versions consistent.

Setup and sources

Official setup: code.claude.com/docs/en/quickstart

Codex CLI: developers.openai.com/codex/cli/

Command reference: developers.openai.com/codex/cli/reference

Original video: youtube.com/watch?v=ucer2chlfM8 (03:30-04:57)

Full kit: markkashef.gumroad.com/l/use-both-claude-codex-workflow-kit

Verification note

Command flags checked against installed Codex help on October 10, 2026. This guide was reviewed for content and syntax; no fresh end-to-end model run is claimed. Model availability and account policies vary.